

1. Sample Programs

2. Write a OpenMP program to sort an array on n elements using both sequential and parallel merge sort (using Selection) Record the difference in execution time.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>
#define SIZE 100000
// ----- MERGE FUNCTION
void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    i = 0; j = 0; k = left;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
    free(L);
    free(R);
}
// ----- SERIAL MERGE SORT
void serialMergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = left + (right - left)/2;
        serialMergeSort(arr, left, mid);
        serialMergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
void parallelMergeSort(int arr[], int left, int right, int depth)
{
    {
```

```

if (left < right)
{
int mid = left + (right - left) / 2;
if (depth <= 4)
{
#pragma omp parallel sections
{
#pragma omp section
parallelMergeSort(arr, left, mid, depth + 1);
#pragma omp section
parallelMergeSort(arr, mid + 1, right, depth + 1);
}
}
else
{
// Switch to serial to avoid too many threads
serialMergeSort(arr, left, mid);
serialMergeSort(arr, mid + 1, right);
}
merge(arr, left, mid, right);
}
}
// ----- MAIN FUNCTION
int main()
{
int *arr_serial = (int *)malloc(SIZE * sizeof(int));
int *arr_parallel = (int *)malloc(SIZE * sizeof(int));
// Initialize both arrays with the same random values
for (int i = 0; i < SIZE; i++)
{
int val = rand() % 100000;
arr_serial[i] = val;
arr_parallel[i] = val;
}
// ----- SERIAL MERGE SORT
clock_t start_serial = clock();
serialMergeSort(arr_serial, 0, SIZE - 1);
clock_t end_serial = clock();
double time_serial = (double)(end_serial - start_serial) / CLOCKS_PER_SEC;
// ----- PARALLEL MERGE SORT
clock_t start_parallel = clock();
parallelMergeSort(arr_parallel, 0, SIZE - 1, 0);
clock_t end_parallel = clock();
double time_parallel = (double)(end_parallel - start_parallel) / CLOCKS_PER_SEC;

```

```
// ----- OUTPUT -----
printf("Serial Merge Sort Time : %.6f seconds \n", time_serial);
printf("Parallel Merge Sort Time : %.6f seconds \n", time_parallel);
free(arr_serial);
free(arr_parallel);
return 0;
}
```

3. Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of the parallelized for loop are executed by which thread.

For example, if there are two threads and four iterations, the output might be the following:

- a. Thread 0 : Iterations 0 – 1**
- b. Thread 1 : Iterations 2 – 3**

```
#include<stdio.h>
#include<omp.h>
int main()
{
    int n = 16,thread;
    printf("\n Enter the number of tasks");
    scanf("%d",&n);
    printf("\n Enter the number of threads");
    scanf("%d",&thread);
    omp_set_num_threads(thread);
    printf("\n ----- \n");
    #pragma omp parallel for schedule(static, 2)
    for (int i = 0; i < n; i++)
    {
        printf("Thread %d executes iteration %d\n", omp_get_thread_num(), i);
    }
    return 0;
}
```

4. Write a OpenMP program to calculate n Fibonacci numbers using tasks.

```
#include <stdio.h>
#include <omp.h>

// Function to calculate Fibonacci numbers recursively (serial)
void fib_serial(int n) {
    int fib_numbers[n+1];
```

```

    fib_numbers[0] = 0;
    fib_numbers[1] = 1;
    printf("Serial Fibonacci numbers: ");
    printf("%d %d ", fib_numbers[0], fib_numbers[1]);
    for (int i = 2; i <= n; i++) {
        fib_numbers[i] = fib_numbers[i-1] + fib_numbers[i-2];
        printf("%d ", fib_numbers[i]);
    }
    printf("\n");
}

// Function to calculate Fibonacci numbers recursively using tasks (parallel)
int fib_parallel(int n) {
    if (n <= 1) return n;
    int x, y;
    #pragma omp task shared(x)
    x = fib_parallel(n-1);
    #pragma omp task shared(y)
    y = fib_parallel(n-2);
    #pragma omp taskwait
    return x + y;
}

void calculate_fib_parallel(int n) {
    int fib_numbers[n+1];
    fib_numbers[0] = 0;
    fib_numbers[1] = 1;
    printf("Parallel Fibonacci numbers: %d %d ", fib_numbers[0], fib_numbers[1]);
    #pragma omp parallel
    {
        #pragma omp single
        for (int i = 2; i <= n; i++) {
            #pragma omp task shared(fib_numbers)
            fib_numbers[i] = fib_parallel(i);
            #pragma omp taskwait
            printf("%d ", fib_numbers[i]);
        }
    }
    printf("\n");
}

int main() {
    int n = 10; // Calculate first 10 Fibonacci numbers

    // Serial calculation
    double start_time = omp_get_wtime();

```

```

    fib_serial(n);
    double end_time = omp_get_wtime();
    printf("Time taken for serial calculation: %f seconds\n", end_time - start_time);

    // Parallel calculation using tasks
    start_time = omp_get_wtime();
    calculate_fib_parallel(n);
    end_time = omp_get_wtime();
    printf("Time taken for parallel calculation: %f seconds\n", end_time - start_time);

    return 0;
}

```

5. Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.

```

#include <stdio.h>
#include <omp.h>
#include <time.h>
int is_prime(int n)
{
    if (n < 2) return 0;
    for (int i = 2; i*i<= n; i++)
        if (n % i == 0) return 0;
    return 1;
}
int main()
{
    long n = 100000000;
    time_t start,end;
    double cpu_time;
    printf("\n the range of numbers is 1 to %ld\n",n);
    printf("\n-----\n");
    // Serial Execution
    start=clock();
    for (int i = 1; i<= n; i++)
    {
        is_prime(i);
    }
    end = clock();
    cpu_time = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf(" Time to compute prime numbers serially: %f\n",cpu_time);
    // Parallel Execution
    start=clock();

```

```

#pragma omp parallel for
for (int i = 1; i <= n; i++)
{
    is_prime(i);
}
end = clock();
cpu_time = ((double) (end - start)) / CLOCKS_PER_SEC;
printf("Time to compute prime numbers Parallel: %f\n",cpu_time);
return 0;
}

```

6. Write a MPI Program to demonstration of MPI_Send and MPI_Recv.

```

#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int number;
    // Initialize the MPI environment
    MPI_Init(&argc, &argv);
    // Get the number of processes
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Get the rank of the process
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (size < 2)
    {
        if (rank == 0)
        {
            printf("This program requires at least 2 processes.\n");
        }
        MPI_Finalize();
        return 0;
    }
    if (rank == 0)
    {
        // Process 0 sends a number to Process 1
        number = 42;
        printf("Process 0 is sending number %d to Process 1\n", number);
        MPI_Send(&number, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }
}

```

```

}

else if (rank == 1)
{
    // Process 1 receives a number from Process 0
    MPI_Recv(&number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
    MPI_STATUS_IGNORE);
    printf("Process 1 received number %d from Process 0\n", number);
}
// Finalize the MPI environment
MPI_Finalize();
return 0;
}

```

7. Write a MPI Program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.

```

#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, data_send, data_recv;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    data_send = rank;

    if (rank == 0)
    {
        MPI_Send(&data_send, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&data_recv, 1, MPI_INT, 1, 0, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);
    }
    else if (rank == 1)
    {
        MPI_Send(&data_send, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Recv(&data_recv, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);
    }
}

```

```

}

printf("Process %d received %d\n", rank, data_recv);
MPI_Finalize();
return 0;
}

```

8. Write a MPI Program to demonstration of Broadcast operation.

```

#include <stdio.h>

#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, data = 0;

    MPI_Init(&argc, &argv);    // Initialize the MPI environment
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Get the rank of the process
    if (rank == 0)
        data = 100;           // Root process sets the data
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD); // Broadcast data from root to all
    printf("Process %d received data: %d\n", rank, data); // All processes print the data
    MPI_Finalize();           // Finalize the MPI environment
    return 0;
}

```

9. Write a MPI Program demonstration of MPI_Scatter and MPI_Gather

```

#include <stdio.h>

#include <mpi.h>

int main(int argc, char** argv)
{
    int rank, size, send_data[4] = {10, 20, 30, 40}, recv_data;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

```

```

MPI_Scatter(send_data, 1, MPI_INT, &recv_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
printf("Process %d received: %d\n", rank, recv_data);

recv_data += 1;
MPI_Gather(&recv_data, 1, MPI_INT, send_data, 1, MPI_INT, 0, MPI_COMM_WORLD);
if (rank == 0)
{
    printf("Gathered data: ");
    for (inti = 0; i < size; i++)
        printf("%d ", send_data[i]);
    printf("\n");
}
MPI_Finalize();
return 0;
}

```

10. Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD)

```

#include <stdio.h>
#include <mpi.h>
int main(int argc, char** argv)
{
    int rank, value, sum, max;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    value = rank + 1;
    MPI_Reduce(&value, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    if (rank == 0)
        printf("Sum using Reduce: %d\n", sum);
}

```

```
MPI_Allreduce(&value, &max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);  
printf("Max using Allreduce (rank %d): %d\n", rank, max);  
MPI_Finalize();  
return 0;  
}
```